



المعهد الوطني للبريد والهواصالات
የኢትዮጵያ ፖስታና ቴሌኮሙኒኬሽን ቢሮ
Institut National des Postes et Télécommunications

INSITUT NATIONAL DES POSTES ET TELECOMMUNICATIONS

BIG DATA – INE2 2026
REAL-TIME STREAMING PIPELINE
RAPPORT

Lab 3 - Spark Streaming & Kafka

Realisé par :
Alae Eddine JAHID
Anass FAJOU

Encadrant :
Pr. Hamid EL GHAZI

26 avril 2026

Table des matières

1	Introduction	3
2	PART 1 – Environment Setup	3
2.1	Q1 : Rôles de chaque composant	3
2.2	Q2 : Installation et démarrage de l’environnement	4
2.3	Q3 : Vérification du bon fonctionnement	5
2.4	Q4 : Création du topic Kafka <code>financial_data</code>	6
3	PART 2 – Data Ingestion (Spark → Kafka)	8
3.1	Environnement de développement : Jupyter Lab	8
3.2	Q5 : Choix de l’API financière et champs pertinents	8
3.3	Q6 : Job Spark qui récupère les données périodiquement	9
3.4	Q7 : Conversion au format JSON et envoi à Kafka	10
3.5	Q8 : Structure d’un message Kafka	11
3.6	Q9 : Tolérance aux pannes et cohérence des données	11
4	PART 3 – Stream Processing (Kafka → Spark)	12
4.1	Q10 : Lecture du topic Kafka avec Spark Structured Streaming	12
4.2	Q11 : Schéma appliqué aux données JSON entrantes	12
4.3	Q12 : Transformations de flux	13
4.4	Q13 : Différence entre batch et Structured Streaming	15
4.5	Q14 : Gestion des données en retard et reprise après panne	15
5	PART 4 – Data Storage (Spark → HDFS)	16
5.1	Q15 : Format de fichier choisi – Parquet	16
5.2	Q16 : Architecture Medallion	17
5.3	Q17 : Stratégie de partitionnement	18
5.4	Q18 : Avantages du partitionnement et des formats colonnaires	19
6	PART 5 – Kafka Streaming Behavior	20
6.1	Q19 : Gestion des offsets et consumer groups	20
6.2	Q20 : Simulation de nouvelles données entrantes	20
6.3	Q21 : Réactivité et latence du pipeline	21
6.4	Q22 : Scénarios de panne	21
6.5	Note opérationnelle : reproductibilité de Kafka entre redémarrages	22
7	PART 6 – Visualization with Superset	23
7.1	Q23 : Exposition des données HDFS via Spark Thrift Server	23
7.2	Q24 : Configuration de Superset et création des datasets	25
7.3	Q25 : Construction des dashboards	26
7.4	Q26 : Choix du type de graphique selon l’analyse	27
7.5	Note opérationnelle : ordre d’exécution pour reproduire la Part 6	27

8	PART 7 – Multidimensional Analysis (Advanced)	27
8.1	Q27 : Prix moyen par symbole et par jour	28
8.2	Q28 : Symbole le plus volatile	28
8.3	Q29 : Heures de pointe (peak trading hours)	29
8.4	Q30 : ROLLUP et CUBE	29
8.5	Q31 : Modélisation en étoile (star schema)	30
8.6	Note opérationnelle : MSCK REPAIR pour les tables partitionnées	31
9	Conclusion	31

1. Introduction

L'objectif de ce lab est de concevoir et mettre en œuvre un pipeline d'analytique financière **temps réel** allant d'une API de données de marché jusqu'à un tableau de bord interactif. Le flux logique est le suivant :

API financière → Spark (Producer) → Kafka → Spark (Consumer) → HDFS → Superset

Contrairement au Lab 2 qui effectuait une extraction **batch** depuis MySQL, ce lab introduit le paradigme **streaming** : les données arrivent en continu, sont traitées au fil de l'eau par Spark Structured Streaming, et sont persistées sur HDFS selon la même architecture Medallion (Bronze / Silver / Gold) que nous avons utilisée précédemment. La source de données retenue est l'API publique REST de **Binance** (`api.binance.com/api/v3/ticker/24hr`) qui expose les données de marché crypto en JSON sans authentification ni inscription préalable.

Dépôt du code. L'intégralité du projet (`docker-compose.yml`, notebooks, scripts, rapport) est disponible publiquement à :

<https://github.com/Alae-J/labs-3-spark-streaming-kafka>

2. PART 1 – Environment Setup

2.1. Q1 : Rôles de chaque composant

Le pipeline repose sur quatre briques technologiques complémentaires, chacune ayant un rôle précis :

- **Kafka** – *Broker de messages* distribué de type publish/subscribe. Il découple les producteurs (l'ingesteur d'API) des consommateurs (le job Spark). Les messages sont **durables**, **ordonnés** au sein d'une partition, et **rejouables** à partir d'un offset antérieur. Kafka joue aussi le rôle de tampon qui absorbe les pics de producteur et permet aux consommateurs de rattraper leur retard.
- **Spark Streaming (Structured Streaming)** – *Moteur de traitement de flux* distribué. Il lit Kafka en micro-batches, applique des transformations DataFrame/SQL (filtrage, agrégations fenêtrées, jointures, moyennes mobiles) et écrit les résultats vers un sink (HDFS, un autre topic Kafka, une console). C'est la couche de *calcul* du pipeline.
- **HDFS** – *Système de fichiers distribué* servant de *data lake*. Il stocke durablement les trois couches Medallion (Bronze, Silver, Gold) en Parquet. C'est le système de référence interrogé ensuite par Superset.
- **Superset** – *Plateforme de Business Intelligence*. Elle se connecte à un moteur SQL (Spark SQL, Trino, Hive), permet d'enregistrer des datasets, de créer des graphiques (séries temporelles, barres, moyennes mobiles) et de les combiner en tableaux de bord. C'est la couche de *présentation*.

2.2. Q2 : Installation et démarrage de l'environnement

L'ensemble de l'environnement est déclaré dans un fichier `docker-compose.yml`. Par rapport au Lab 2, nous avons ajouté quatre services :

- **Zookeeper** – service de coordination requis par Kafka (élection du leader, métadonnées des partitions).
- **Kafka broker** – le broker de messages (`confluentinc/cp-kafka:7.5.0`).
- **Kafka UI** – interface web pour inspecter les topics, partitions, offsets et consumer groups.
- **Superset** – serveur BI (`apache/superset:3.1.1`) initialisé automatiquement avec un compte admin.

Tous les noms de conteneurs sont préfixés par `lab3-` afin de ne pas entrer en conflit avec les labs précédents, et l'ensemble partage un réseau Docker dédié `lab3`.

```
version: "3.8"

services:
  namenode:
    image: bde2020/hadoop-namenode:2.0.0-hadoop3.2.1-java8
    container_name: lab3-namenode
    hostname: namenode
    ports: ["9870:9870"]

  datanode1:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: lab3-datanode1

  datanode2:
    image: bde2020/hadoop-datanode:2.0.0-hadoop3.2.1-java8
    container_name: lab3-datanode2

  zookeeper:
    image: confluentinc/cp-zookeeper:7.5.0
    container_name: lab3-zookeeper
    ports: ["2181:2181"]
    environment:
      ZOOKEEPER_CLIENT_PORT: 2181

  kafka:
    image: confluentinc/cp-kafka:7.5.0
    container_name: lab3-kafka
    depends_on: [zookeeper]
    ports: ["9092:9092", "29092:29092"]
    environment:
      KAFKA_BROKER_ID: 1
      KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
      KAFKA_ADVERTISED_LISTENERS: >-
        PLAINTEXT://kafka:9092,PLAINTEXT_HOST://localhost:29092
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
```

```

kafka-ui:
  image: provectuslabs/kafka-ui:latest
  container_name: lab3-kafka-ui
  ports: ["8085:8080"]

spark-master:
  image: apache/spark:3.5.0
  container_name: lab3-spark-master
  ports: ["7077:7077", "8080:8080"]

spark-worker1:
  image: apache/spark:3.5.0
  container_name: lab3-spark-worker1

spark-worker2:
  image: apache/spark:3.5.0
  container_name: lab3-spark-worker2

spark-client:
  image: apache/spark:3.5.0
  container_name: lab3-spark-client
  volumes: [./scripts:/opt/scripts]
  ports: ["4040:4040"]

superset:
  image: apache/superset:3.1.1
  container_name: lab3-superset
  ports: ["8088:8088"]

```

Listing 1 – docker-compose.yml (extrait)

Deux listeners Kafka sont définis :

- **kafka:9092** – utilisé par les conteneurs sur le réseau **lab3** (jobs Spark, Superset).
- **localhost:29092** – utilisé depuis la machine hôte (producteur Python exécuté en dehors de Docker).

Lancement de la stack complète :

```
docker-compose up -d
```

Interfaces web disponibles après démarrage :

Service	URL
HDFS NameNode	http://localhost:9870
Spark Master	http://localhost:8080
Spark Application	http://localhost:4040
Kafka UI	http://localhost:8085
Superset	http://localhost:8088 (admin / admin)

2.3. Q3 : Vérification du bon fonctionnement

Chaque service est vérifié indépendamment avant de passer à la suite du lab.

Vérifier que Kafka fonctionne :

```
docker exec lab3-kafka kafka-broker-api-versions \
--bootstrap-server kafka:9092
```

Le broker répond avec la liste de ses API supportées (Produce, Fetch, Metadata...). Une erreur de connexion signalerait que Zookeeper ou Kafka ne sont pas disponibles. L'interface Kafka UI (localhost:8085) affiche également l'état du cluster.

Vérifier que HDFS est accessible :

```
docker exec lab3-namenode hdfs dfsadmin -report
```

La sortie attendue confirme la présence de **2 DataNodes vivants** et la capacité configurée :

```
Configured Capacity: 259795787776 (241.95 GB)
DFS Used: 49152 (48 KB)
Live datanodes (2):
  Name: 172.X.X.X:9866 (datanode1)
  Name: 172.X.X.X:9866 (datanode2)
```

L'UI HDFS (localhost:9870) dans l'onglet *Datanodes* doit lister *datanode1* et *datanode2* avec le statut *In service*.

Vérifier que Spark est correctement configuré :

```
docker exec lab3-spark-client bash -c \
"echo 'println(spark.version)' | \
/opt/spark/bin/spark-shell \
--master spark://spark-master:7077"
```

La sortie attendue contient :

```
Spark context available as 'sc'
(master = spark://spark-master:7077, app id = app-20260423...)
version 3.5.0
Using Scala version 2.12.18, Java 11.0.20.1
```

L'UI du Spark Master (localhost:8080) doit également afficher les **2 Workers** enregistrés.

Vérifier que Superset est opérationnel :

```
curl -s -o /dev/null -w "%{http_code}\n" \
http://localhost:8088/health
# -> 200
```

La page de connexion est accessible à localhost:8088 avec les identifiants *admin / admin* créés automatiquement par le conteneur au premier démarrage.

2.4. Q4 : Création du topic Kafka *financial_data*

Le topic Kafka est créé via l'outil en ligne de commande *kafka-topics* livré avec l'image Confluent :

```
docker exec lab3-kafka kafka-topics \
--bootstrap-server kafka:9092 \
```



```
--create \
--topic financial_data \
--partitions 3 \
--replication-factor 1 \
--config retention.ms=604800000
```

Choix du nombre de partitions. Nous avons retenu **3 partitions** pour les raisons suivantes :

1. **Plafond de parallélisme** – le nombre de partitions fixe le nombre maximal de consommateurs *actifs* au sein d'un même consumer group. Plus il y a de partitions, plus on peut paralléliser la consommation.
2. **Alignement avec la couche de calcul** – notre cluster Spark dispose de 2 workers ; 3 partitions laissent à Spark une marge de parallélisation sans pour autant fragmenter excessivement les données.
3. **Alignement avec la cardinalité des symboles** – nous comptons streamer un petit panier de symboles (BTCUSDT, ETHUSDT, SOLUSDT). En utilisant le symbole comme **clé** de message, Kafka garantit automatiquement que tous les ticks d'un même symbole atterrissent sur la *même* partition, ce qui préserve leur ordre temporel.
4. **Pas de sur-provisionnement** – sur un broker unique, une seule partition sérialiserait toutes les lectures tandis qu'un nombre élevé (30+) ajouterait de l'overhead de métadonnées sans gain réel. 3 est un bon compromis pédagogique.

Le facteur de réplication est de **1** car nous n'avons qu'un seul broker. En production on utiliserait typiquement `replication-factor=3` réparti sur 3 brokers pour tolérer la perte d'un nœud. La rétention est fixée à **7 jours** (604800000 ms), largement suffisante pour un lab et permettant de rejouer le flux si nécessaire.

Vérification du topic créé :

```
docker exec lab3-kafka kafka-topics \
--bootstrap-server kafka:9092 \
--describe --topic financial_data
```

Sortie obtenue :

```
Topic: financial_data PartitionCount: 3 ReplicationFactor: 1
Partition: 0 Leader: 1 Replicas: 1 Isr: 1
Partition: 1 Leader: 1 Replicas: 1 Isr: 1
Partition: 2 Leader: 1 Replicas: 1 Isr: 1
```

Test end-to-end : nous validons la plomberie Kafka en produisant un message puis en le consommant depuis le début :

```
# Produire un message
echo '{"test":"hello from lab3"}' | \
  docker exec -i lab3-kafka kafka-console-producer \
    --bootstrap-server kafka:9092 --topic financial_data

# Consommer le message
docker exec lab3-kafka kafka-console-consumer \
  --bootstrap-server kafka:9092 --topic financial_data \
  --from-beginning --max-messages 1
```

```
# -> {"test":"hello from lab3"}
# -> Processed a total of 1 messages
```

Le message est bien récupéré côté consommateur, confirmant que la chaîne Zookeeper → Kafka → client fonctionne de bout en bout.

3. PART 2 – Data Ingestion (Spark → Kafka)

3.1. Environnement de développement : Jupyter Lab

Pour faciliter l'expérimentation itérative, nous avons ajouté à la stack un service **Jupyter Lab** basé sur l'image officielle `jupyter/pyspark-notebook:spark-3.5.0`. Toutes les opérations des parties 2 et suivantes sont pilotées depuis des notebooks `.ipynb` montés dans le conteneur :

```
jupyter:
  image: jupyter/pyspark-notebook:spark-3.5.0
  container_name: lab3-jupyter
  depends_on: [spark-master, kafka, namenode]
  ports: ["8888:8888"]
  volumes:
    - ./notebooks:/home/jovyan/work/notebooks
    - ./scripts:/home/jovyan/work/scripts
  environment:
    JUPYTER_TOKEN: lab3
  command: >
    bash -c "pip install --quiet --user requests kafka-python
      findspark &&
      start-notebook.sh --ServerApp.token='lab3' --ServerApp
        .password=''"
```

Listing 2 – Extrait docker-compose.yml – service Jupyter

Jupyter Lab est accessible à `http://localhost:8888` avec le token `lab3`. Le notebook `notebooks/02_producer_binance.ipynb` contient le producteur détaillé ci-après.

Note technique importante : le conteneur Jupyter utilise Python 3.11 (via conda), alors que les Spark Workers (`apache/spark:3.5.0`) utilisent Python 3.8. PySpark exige que le driver et les executors partagent la même version mineure de Python. Par conséquent, les notebooks s'exécutent en **mode local** (`master="local[*]"`) dans le conteneur Jupyter : le driver et les executors tournent dans le même processus, sans conflit. Le cluster distribué reste disponible pour les jobs lourds soumis via `spark-submit` depuis le conteneur `lab3-spark-client`, qui utilise Python 3.8 compatible avec les workers. Pour les micro-batches de ce lab (5 lignes par itération), le mode local est en pratique le bon choix.

3.2. Q5 : Choix de l'API financière et champs pertinents

Nous avons retenu l'**API publique de Binance** (`api.binance.com` et son miroir `data-api.binance.vision`) pour les raisons suivantes :

- **Sans authentication** – les endpoints de données de marché publiques ne requièrent ni clé API, ni inscription, ni carte bancaire. Idéal en contexte étudiant.

- **Limites de taux généreuses** – plusieurs milliers de requêtes par minute par IP, largement suffisant pour polluer à 5 s.
- **Marché crypto 24/7** – contrairement aux bourses actions, le marché crypto est toujours ouvert, donc le pipeline peut être démontré à tout moment.
- **JSON propre** – réponses directement exploitables par Spark.

L'endpoint utilisé est `GET /api/v3/ticker/24hr?symbol=BTCUSDT` qui renvoie une structure JSON riche. Nous en extrayons 4 champs suffisants pour toute l'analyse :

Champ	Source Binance	Justification
symbol	symbol	identifiant de l'actif (clé Kafka)
price	lastPrice	dernier prix transactionnel
volume	volume	volume échangé sur 24h
timestamp	closeTime	horodatage en millisecondes (epoch)

Le panier de symboles suivi est [BTCUSDT, ETHUSDT, SOLUSDT, BNBUSDT, ADAUSDT] – un échantillon représentatif de cryptos majeures avec des ordres de grandeur de prix très différents (de \$0.55 pour ADA à \$77000 pour BTC), ce qui permettra d'illustrer le besoin de normalisation lors des agrégations de Part 3.

3.3. Q6 : Job Spark qui récupère les données périodiquement

Le notebook `02_producer_binance.ipynb` définit un schéma Spark strict et une fonction `fetch_ticker` qui effectue un appel REST pour un symbole :

```
from pyspark.sql.types import StructType, StructField, \
    StringType, DoubleType, LongType

SCHEMA = StructType([
    StructField("symbol",      StringType(), False),
    StructField("price",      DoubleType(), False),
    StructField("volume",     DoubleType(), False),
    StructField("timestamp", LongType(), False),
])

def fetch_ticker(symbol: str) -> dict:
    r = requests.get(
        "https://api.binance.com/api/v3/ticker/24hr",
        params={"symbol": symbol}, timeout=5)
    r.raise_for_status()
    d = r.json()
    return {
        "symbol":      d["symbol"],
        "price":       float(d["lastPrice"]),
        "volume":      float(d["volume"]),
        "timestamp":   int(d["closeTime"]),
    }
```

Listing 3 – Schéma et helper REST

La session Spark est créée avec le connecteur Kafka en tant que dépendance Maven – Spark le télécharge automatiquement au premier appel puis le met en cache dans `~/.ivy2/` :

```
import findspark; findspark.init()
from pyspark.sql import SparkSession

spark = (SparkSession.builder
        .appName("BinanceToKafkaProducer")
        .master("local[*]")
        .config("spark.jars.packages",
              "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.0")
        .getOrCreate())
```

3.4. Q7 : Conversion au format JSON et envoi à Kafka

L'API Kafka de Spark attend un DataFrame à deux colonnes **key** et **value** (chaque valeur étant une chaîne ou un tableau d'octets). Nous convertissons donc chaque enregistrement en JSON via la fonction `to_json(struct(...))` et utilisons le **symbole** comme clé pour assurer que tous les ticks d'un même actif atterrissent sur la même partition (préservation de l'ordre par symbole) :

```
from pyspark.sql.functions import col, struct, to_json
import time

while True:
    records = [fetch_ticker(s) for s in SYMBOLS]
    df = spark.createDataFrame(records, schema=SCHEMA)

    out = df.select(
        col("symbol").cast("string").alias("key"),
        to_json(struct("symbol", "price", "volume", "timestamp"))
        .alias("value"),
    )

    (out.write
     .format("kafka")
     .option("kafka.bootstrap.servers", "kafka:9092")
     .option("topic", "financial_data")
     .save())

    time.sleep(5)
```

Listing 4 – Boucle de polling et envoi à Kafka

Test end-to-end réussi. Après une itération réelle, nous avons consommé le topic pour vérifier la réception des messages (sortie tronquée) :

```
ETHUSDT | {"symbol":"ETHUSDT","price":2313.57,
          "volume":372630.82,"timestamp":1776940116013}
ADAUSDT | {"symbol":"ADAUSDT","price":0.2455,
          "volume":1.14054E8,"timestamp":1776940117003}
BTCUSDT | {"symbol":"BTCUSDT","price":77379.31,
          "volume":18422.16,"timestamp":1776940115002}
BNBUSDT | {"symbol":"BNBUSDT","price":631.52,
```

```

      "volume":127307.82,"timestamp":1776940117008}
SOLUSDT | {"symbol":"SOLUSDT","price":85.38,
      "volume":2595530.01,"timestamp":1776940116625}

```

La clé à gauche (ETHUSDT, ADAUSDT, ...) correspond bien au symbole ; chaque valeur est un document JSON conforme au schéma.

3.5. Q8 : Structure d'un message Kafka

Un message Kafka est une structure en 6 champs :

Champ	Rôle
key	Clé (bytes). Détermine la partition via hachage. Chez nous = symbole.
value	Payload métier (bytes). Chez nous = JSON sérialisé.
topic	Nom du topic cible (<code>financial_data</code>).
partition	Index de la partition (0, 1, ou 2). Attribué par Kafka en fonction de la clé.
offset	Position séquentielle dans la partition. Immuable.
timestamp	Horodatage créé par le producteur ou par le broker.

Les **headers** optionnels (clé-valeur, comme des headers HTTP) peuvent transporter des méta-données techniques (version du schéma, correlation-id) ; nous ne les utilisons pas dans ce lab.

3.6. Q9 : Tolérance aux pannes et cohérence des données

Tolérance aux pannes. Elle repose sur trois mécanismes complémentaires :

- **Réplication Kafka** – en production, chaque partition est répliquée sur N brokers (`replication-factor=3` typique). Si un broker tombe, un suiveur *in-sync* prend le relais comme leader. Notre lab n'a qu'un broker (`replication-factor=1`), donc ce filet de sécurité est désactivé ; on le documente sans le démontrer.
- **Durabilité côté producteur** – le connecteur Kafka de Spark attend par défaut l'`ack` du broker avant de considérer l'écriture comme réussie. Une configuration plus stricte passerait par `kafka.acks="all"` (attente de l'`ack` de tous les replicas in-sync) et `kafka.retries=N` (réémission automatique en cas d'erreur transitoire), à spécifier dans les options du `.write.format("kafka")`.
- **Commit d'offsets côté consommateur** – le job consommateur Spark (Part 3) s'appuiera sur un `checkpointLocation` HDFS pour persister les offsets consommés et les garanties de progression ; en cas de crash, le job reprend exactement là où il s'était arrêté.

Cohérence des données. Elle est assurée par :

- **Ordre par clé** – Kafka garantit l'ordre des messages *au sein d'une même partition*. En utilisant `symbol` comme clé, tous les ticks d'un même actif sont linéarisés dans le temps.

- **Schéma strict** – notre `StructType` rejette toute ligne mal formée (type incorrect, champ manquant) dès l'écriture, ce qui évite qu'un tick corrompu pollue le topic.
- **Sémantique at-least-once** – par défaut, le producteur peut réémettre un message en cas de doute sur la réception de l'ack, ce qui peut produire des doublons ; ceux-ci sont éliminés en Part 3 via `dropDuplicates` sur `(symbol, timestamp)`. Pour `exactly-once`, Kafka offre la propriété `enable.idempotence=true` côté producteur et les transactions côté consommateur/producteur, que nous n'activons pas dans ce lab mais mentionnons pour complétude.

4. PART 3 – Stream Processing (Kafka → Spark)

Tout le traitement de flux est contenu dans le notebook `notebooks/03_stream_processor.ipynb`. Il s'exécute en parallèle du producteur (Part 2) : le producteur continue d'alimenter le topic pendant que le consommateur en lit le flux en continu.

4.1. Q10 : Lecture du topic Kafka avec Spark Structured Streaming

Spark Structured Streaming traite les sources Kafka comme des tables infinies : chaque nouveau message ajoute une ligne. La lecture se fait via `spark.readStream.format("kafka")` avec les mêmes options qu'en batch, plus `startingOffsets` qui contrôle le point de départ :

```
raw = (spark.readStream
      .format("kafka")
      .option("kafka.bootstrap.servers", "kafka:9092")
      .option("subscribe", "financial_data")
      .option("startingOffsets", "latest")
      .load())
```

Listing 5 – Lecture streaming de Kafka

Le `DataFrame` `raw` a un schéma imposé par Kafka : `key (binary)`, `value (binary)`, `topic`, `partition`, `offset`, `timestamp`, `timestampType`. La valeur métier se trouve dans `value` sous forme d'octets – il faut la décoder et parser le JSON.

4.2. Q11 : Schéma appliqué aux données JSON entrantes

Structured Streaming **n'infère pas** de schéma sur un flux (contrairement à `spark.read.json`). Nous devons le fournir explicitement – c'est le même que celui utilisé par le producteur, ce qui garantit une correspondance parfaite :

```
from pyspark.sql.types import StructType, StructField, \
    StringType, DoubleType, LongType
from pyspark.sql.functions import col, from_json, from_unixtime

SCHEMA = StructType([
    StructField("symbol", StringType(), False),
    StructField("price", DoubleType(), False),
    StructField("volume", DoubleType(), False),
    StructField("timestamp", LongType(), False),
```

```

))

parsed = (raw
  .selectExpr("CAST(value AS STRING) AS json_str")
  .select(from_json(col("json_str"), SCHEMA).alias("d"))
  .select("d.symbol", "d.price", "d.volume", "d.timestamp")
  .withColumn("event_time",
    from_unixtime(col("timestamp") / 1000).cast("timestamp")))

```

La fonction `from_json` tente de parser chaque `value` selon le schéma. Si une ligne est mal formée, les champs sont remplis avec `null`, ce qui permet de la filtrer ensuite. On ajoute une colonne `event_time` de type `Timestamp` construite à partir du champ `timestamp` du producteur (en millisecondes epoch) – indispensable pour les fenêtres temporelles et les watermarks.

4.3. Q12 : Transformations de flux

Trois transformations obligatoires par l'énoncé sont implantées.

Transformation 1 – Filtrage des valeurs anormales. Deux garde-fous sont appliqués :

1. **Cordons de sécurité basiques** – rejet des prix négatifs ou nuls, volumes négatifs.
2. **Bandes par symbole** – chaque symbole connu a un prix d'ancrage approximatif (ANCHOR); un tick dont le prix sort de l'intervalle $[0.3 \times \text{anchor}, 3.0 \times \text{anchor}]$ est considéré comme aberrant. Ces bandes larges laissent passer la volatilité normale du marché tout en capturant les vraies erreurs (spike $\times 10$, crash $\times 0.1$).

```

from pyspark.sql.functions import lit, when

ANCHOR = {"BTCUSDT":77000.0, "ETHUSDT":2300.0,
          "SOLUSDT":85.0, "BNBUSDT":630.0, "ADAUSDT":0.25}
LOWER, UPPER = 0.3, 3.0

anchor_expr = None
for sym, px in ANCHOR.items():
    branch = when(col("symbol") == sym, lit(px))
    anchor_expr = branch if anchor_expr is None \
        else anchor_expr.when(col("symbol") == sym, lit(px))
anchor_expr = anchor_expr.otherwise(lit(None))

clean = (parsed
  .withColumn("anchor_px", anchor_expr)
  .filter(col("price") > 0)
  .filter(col("volume") >= 0)
  .filter(col("anchor_px").isNull() |
    ((col("price") >= col("anchor_px") * LOWER) &
     (col("price") <= col("anchor_px") * UPPER)))
  .drop("anchor_px"))

```

Listing 6 – Filtre des valeurs anormales

Transformation 2 – Moyenne mobile. Fenêtre glissante de 1 minute avançant toutes les 10 secondes, groupée par symbole. Chaque tick contribue à plusieurs fenêtres simultanément (6 fenêtres actives à tout instant par symbole). Un **watermark** de 2 minutes indique à Spark qu'on peut ignorer les données retardataires au-delà de 2 min.

```
from pyspark.sql.functions import window, avg, count

moving_avg = (clean
    .withWatermark("event_time", "2 minutes")
    .groupBy(
        window(col("event_time"), "1 minute", "10 seconds"),
        col("symbol"))
    .agg(
        avg("price").alias("avg_price"),
        count("*").alias("n_ticks"))
    .select("symbol", "window.start", "window.end",
        "avg_price", "n_ticks"))
```

Listing 7 – Moyenne mobile glissante

Transformation 3 – Agrégation par symbole. Agrégation running (sans fenêtre) depuis le début du flux : pour chaque symbole, min/max/avg du prix, somme des volumes, nombre de ticks.

```
from pyspark.sql.functions import min as _min, max as _max, sum as
_sum

per_symbol = (clean.groupBy("symbol").agg(
    _min("price").alias("min_price"),
    _max("price").alias("max_price"),
    avg("price").alias("avg_price"),
    _sum("volume").alias("total_volume"),
    count("*").alias("n_ticks")))
```

Listing 8 – Agrégation par symbole

Résultat observé. Après 45 s de flux réel (test end-to-end contre Binance), voici une sortie représentative de l'agrégation par symbole (batch 4, sink console) :

symbol	min_price	max_price	avg_price	total_volume	n_ticks
BTCUSDT	77370.0	77393.56	77385.705	110550.96	6
BNBUSDT	631.9	632.07	632.015	765668.00	6
SOLUSDT	85.38	85.41	85.397	15546465.30	6
ADAUSDT	0.2456	0.2457	0.24562	681859467.9	6
ETHUSDT	2312.75	2313.42	2313.183	2247153.67	6

Les 5 symboles sont tous présents, aucune valeur aberrante n'a été détectée dans ce échantillon (le marché était calme), ce qui est le comportement attendu.

4.4. Q13 : Différence entre batch et Structured Streaming

Critère	Batch (<code>spark.read</code>)	Structured Streaming (<code>spark.readStream</code>)
Source	Dataset fini (fichier, table)	Table infinie (Kafka, socket, fichiers qui apparaissent)
Exécution	Une fois, traite toutes les lignes, s'arrête	Continue, traite les nouvelles lignes au fur et à mesure
Schema inference	Optionnel	Obligatoire (fourni par l'utilisateur)
Trigger	N/A	<code>ProcessingTime(n)</code> , <code>Once</code> , <code>Available now</code> , <code>Continuous</code>
Output modes	<code>overwrite</code> , <code>append</code>	<code>append</code> , <code>update</code> , <code>complete</code>
Tolérance aux pannes	Reprise du job entier	Reprise au dernier offset validé via <code>checkpointLocation</code>
Fenêtres temporelles	N/A (ou via <code>groupBy</code> temporel)	Natives (<code>window(...)</code>) avec gestion explicite de l' <i>event time</i> et du watermark

Une conséquence importante : **l'API `DataFrame` est identique** entre les deux modes (`select`, `filter`, `groupBy`, `join...`). Un script peut souvent passer de batch à streaming en ne changeant que `read` en `readStream` et en ajoutant un trigger.

4.5. Q14 : Gestion des données en retard et reprise après panne

Données en retard (late data). Elles arrivent après leur temps événement censé – par exemple à cause d'une latence réseau côté producteur, ou d'un buffer Kafka saturant. Spark traite ce problème via deux mécanismes :

- **Watermark** (`withWatermark("event_time", "2 minutes")`) – indique à Spark le seuil au-delà duquel une donnée est trop ancienne pour être intégrée dans une agrégation. Concrètement, si le dernier `event_time` vu est T , toute nouvelle ligne avec $\text{event_time} < T - 2\text{min}$ est ignorée.
- **State store** – entre le watermark et l'horizon courant, Spark conserve l'état des fenêtres ouvertes. Une donnée late qui tombe dans une fenêtre encore ouverte **met à jour** l'agrégat correspondant (mode `update`), sinon elle est abandonnée.

Le choix du watermark est un compromis : trop court → on perd des données légitimement retardées ; trop long → l'état en mémoire croît sans cesse.

Reprise après panne (fault recovery). Chaque query streaming définit un répertoire `checkpointLocation` où Spark persiste trois types d'information :

1. les **offsets Kafka** consommés (jusqu'où on en est) ;
2. l'**état des agrégats** en cours (pour les queries avec `groupBy`) ;
3. les **métadonnées de commit** de chaque micro-batch.

```
q = (per_symbol.writeStream
    .format("console")
    .outputMode("complete")
    .option("checkpointLocation", "/tmp/lab3_ckpt/per_symbol"))
```

```
.trigger(processingTime="10 seconds")
.queryName("per_symbol_console")
.start()
```

Listing 9 – Query streaming avec checkpoint

Si le job plante (par exemple le conteneur Spark est tué), il suffit de relancer le même script : Spark inspecte le répertoire de checkpoint, reprend aux offsets non commités et reconstitue l'état des agrégats. La garantie offerte est **exactly-once** pour le calcul des agrégats (grâce au state store transactionnel) et **at-least-once** pour la livraison vers un sink non-transactionnel comme la console ou Parquet – ce qui explique pourquoi la déduplication reste une bonne pratique en aval (`dropDuplicates` sur `(symbol, timestamp)`).

En Part 4 nous remplacerons le sink console par un sink **Parquet sur HDFS** pour persister les couches Bronze/Silver/Gold du data lake.

5. PART 4 – Data Storage (Spark → HDFS)

Le notebook `notebooks/04_hdfs_medallion.ipynb` reprend la logique streaming de Part 3 mais remplace les sinks console par **trois sinks Parquet sur HDFS**, chacun correspondant à une couche Medallion. Les trois queries s'exécutent *simultanément* dans la même SparkSession, chacune avec son propre `checkpointLocation` dans HDFS.

Avant le premier démarrage, on crée la racine `/lab3` dans HDFS avec des droits ouverts pour que l'utilisateur `jovyan` du conteneur Jupyter puisse écrire :

```
docker exec lab3-namenode hdfs dfs -mkdir -p \
  /lab3/bronze /lab3/silver /lab3/gold /lab3/checkpoints
docker exec lab3-namenode hdfs dfs -chmod -R 777 /lab3
```

5.1. Q15 : Format de fichier choisi – Parquet

Nous utilisons **Apache Parquet** (avec compression **Snappy** par défaut) pour les trois couches. Les raisons principales :

- **Format colonnaire** – les données sont stockées par colonne plutôt que par ligne. Une requête « moyenne du `price` par `symbol` » ne lit que deux colonnes du fichier, sans charger `volume`, `timestamp`, etc.
- **Compression native** – Snappy donne un bon compromis vitesse/taille ; les mêmes données CSV auraient été ~ 3 à 5 fois plus volumineuses.
- **Schéma embarqué** – types (Double, Long, String...) conservés dans le fichier. Pas de parsing/inférence comme avec CSV.
- **Predicate pushdown** – Spark peut envoyer des filtres au lecteur Parquet qui saute des row-groups entiers sans les décompresser (« `where symbol='BTCUSDT'` »).
- **Evolvable** – on peut ajouter ou retirer des colonnes sans casser les lecteurs existants.

Comparaison rapide avec les alternatives typiques :

Format	Verdict	Commentaire
Parquet	Choisi	Colonnaire, compressé, schéma, push-down.
ORC	Valable	Très proche de Parquet, mieux intégré côté Hive/ACID.
CSV	Rejeté	Non colonnaire, pas de types, pas de compression native.
JSON	Rejeté	Volumineux, pas colonnaire. Idéal pour <i>transport</i> (Kafka), pas pour <i>stockage</i> analytique.

5.2. Q16 : Architecture Medallion

Nous reprenons le pattern Bronze / Silver / Gold déjà utilisé au Lab 2, adapté au contexte streaming :

Couche	Contenu	Niveau de traitement
Bronze	Ticks parses depuis Kafka, tels quels – aucune suppression.	Fidélité totale à la source ; permet le « replay » en cas de changement de règle métier.
Silver	Ticks après le filtre d'outliers de la Part 3 (prix > 0, volume ≥ 0, prix dans $[0.3\times, 3\times]$ de l'ancre).	Données « propres » prêtes à consommer pour des analyses opérationnelles.
Gold	Moyennes mobiles 1-min (glissement 10 s) par symbole, <i>finalisées</i> après passage du watermark.	Agrégats métier directement consommables par Superset ou un notebook d'analyse.

La couche Bronze utilise le sink Parquet en mode **append** sur les ticks bruts. La couche Silver fait pareil sur les ticks filtrés. La couche Gold est légèrement plus subtile : en streaming, l'écriture Parquet ne supporte que le mode **append**, ce qui impose que les agrégats fenêtrés soient *définitifs* avant écriture. Le watermark garantit cette finalisation – une fenêtre $[t, t + 1\text{min}]$ est écrite une et une seule fois lorsque l'horloge événementielle dépasse $t + 1\text{min} + 2\text{min watermark}$.

```
HDFS = "hdfs://namenode:9000/lab3"

q_bronze = (parsed.writeStream
  .format("parquet")
  .option("path", f"{HDFS}/bronze/ticks")
  .option("checkpointLocation", f"{HDFS}/checkpoints/bronze")
  .partitionBy("ingest_date")
  .outputMode("append")
  .trigger(processingTime="20 seconds")
  .start())

q_silver = (clean.writeStream
```

```

    .format("parquet")
    .option("path", f"{HDFS}/silver/ticks")
    .option("checkpointLocation", f"{HDFS}/checkpoints/silver")
    .partitionBy("ingest_date", "symbol")
    .outputMode("append")
    .trigger(processingTime="20 seconds")
    .start()

q_gold = (moving_avg.writeStream
    .format("parquet")
    .option("path",
        f"{HDFS}/gold/moving_avg")
    .option("checkpointLocation",
        f"{HDFS}/checkpoints/gold_moving_avg")
    .partitionBy("window_date")
    .outputMode("append")
    .trigger(processingTime="30 seconds")
    .start())

```

Listing 10 – Les 3 writeStream vers HDFS

5.3. Q17 : Stratégie de partitionnement

Chaque couche a une stratégie de partitionnement adaptée à ses requêtes typiques :

- **Bronze** – partitionné par `ingest_date`. Cas d’usage principal : reprocesser « hier » ou « cette semaine ». Pas de partition par symbole ici – en amont on ne sait pas encore si un tick est valide, et multiplier les partitions ajouterait de la fragmentation inutile.
- **Silver** – partitionné par `ingest_date` puis par `symbol`. La plupart des analyses filtrent sur un symbole précis (« évolution de BTCUSDT sur le 2026-04-23 »). Avoir `symbol=` dans le path permet à Spark de **skipper** entièrement les répertoires des autres symboles.
- **Gold** – partitionné par `window_date`. Les agrégats sont relativement petits et le discriminant dominant reste temporel.

Structure observée dans HDFS après 200 s de flux :

```

/lab3/bronze/ticks/
_spark_metadata/...
ingest_date=2026-04-23/
  part-00000-*.snappy.parquet
  part-00001-*.snappy.parquet
  ...

/lab3/silver/ticks/
_spark_metadata/...
ingest_date=2026-04-23/
  symbol=BTCUSDT/ part-*.snappy.parquet
  symbol=ETHUSDT/ part-*.snappy.parquet
  symbol=SOLUSDT/ part-*.snappy.parquet

```

```

symbol=BNBUSDT/ part-*.snappy.parquet
symbol=ADAUSD/ part-*.snappy.parquet

/lab3/gold/moving_avg/
_spark_metadata/...
window_date=2026-04-23/
part-*.snappy.parquet

```

Vérification côté Spark : la lecture batch des trois couches renvoie bien les volumes attendus :

Couche	Lignes	Commentaire
Bronze	160	tous les ticks parsés (5 symboles × ~32 itérations)
Silver	160	aucun outlier détecté pendant ce test (marché calme)
Gold	5	fenêtres déjà finalisées par le watermark

Note opérationnelle : le problème des petits fichiers. Chaque micro-batch de Structured Streaming crée un nouveau fichier Parquet (~1.8 Ko chacun dans nos tests). Sur le long terme, HDFS préfère des fichiers de ~128 Mo (taille d'un bloc). En production, on planifie un job de **compactage** périodique (quotidien) qui lit les petits fichiers d'une partition et les réécrit en un fichier unique. Ce job est hors scope pour ce lab mais est à prévoir pour un déploiement réel.

5.4. Q18 : Avantages du partitionnement et des formats colonnaires

Partitionnement. Découpe physiquement le jeu de données selon une ou plusieurs colonnes, en créant un répertoire par valeur. Avantages concrets :

- **Partition pruning** – une requête `WHERE ingest_date = '2026-04-23' AND symbol = 'BTCUSD'` va lire *uniquement* le répertoire `ingest_date=2026-04-23/symbol=BTCUSD/`. Les autres sont complètement ignorés. Gain proportionnel au nombre de partitions écartées.
- **Parallélisation naturelle** – chaque partition peut être lue par un worker différent, sans coordination.
- **Rejouabilité** – on peut purger, recalculer ou réimporter une partition (« hier uniquement ») sans toucher au reste.
- **Indépendance des étapes ETL** – un job qui écrit la partition du 23 avril ne bloque pas un job qui lit la partition du 22 avril.

Piège : partitionner sur une colonne à haute cardinalité (par exemple `timestamp` à la milliseconde) produirait des millions de minuscules fichiers et dégraderait les performances. La règle pratique est de choisir une ou deux colonnes naturelles à cardinalité $\leq$ quelques centaines de valeurs distinctes – comme `ingest_date` et `symbol`.

Formats colonnaires (Parquet / ORC). Le stockage colonnaire stocke les valeurs d'une même colonne contiguës sur disque. Avantages :

- **Projection** – une requête qui ne réclame que `symbol`, `price` lit deux colonnes au lieu de toutes. Gain proportionnel à $1 - (\text{colonnes lues} / \text{colonnes totales})$.
- **Compression supérieure** – des valeurs similaires (des prix, des symboles) se compressent beaucoup mieux côte-à-côte que mélangées.

- **Predicate pushdown / statistiques** – Parquet stocke min/max/null_count par row-group. Spark utilise ces stats pour sauter des blocs entiers sans les lire (« tous les `price` de ce bloc sont < 100, inutile de le décompresser si on cherche > 1000 »).
- **Encodages dict/RLE** – Parquet remplace automatiquement les colonnes à faible cardinalité (ex. `symbol`) par un dictionnaire + indices, réduisant drastiquement la taille.

La combinaison **partitionnement** + **colonnaire** + **predicate pushdown** est ce qui rend Spark rapide sur des volumes que des outils row-based (MySQL, JSON, CSV...) peineraient à charger.

6. PART 5 – Kafka Streaming Behavior

Le notebook `notebooks/05_streaming_behavior.ipynb` regroupe trois démonstrations : (i) inspection des offsets et consumer groups, (ii) mesure de la latence end-to-end, (iii) scénarios de panne (broker Kafka, job Spark).

6.1. Q19 : Gestion des offsets et consumer groups

Offsets. Chaque message dans une partition Kafka reçoit un **entier monotone croissant** (0, 1, 2...) appelé *offset*. La position d'un consommateur est entièrement décrite par le triplet (`topic`, `partition`, `offset`). Kafka lui-même ne déplace jamais l'offset : c'est le consommateur qui choisit jusqu'où il a lu et « commit » cette position.

Consumer groups. Un *group ID* est une chaîne arbitraire qui lie plusieurs processus consommateurs entre eux. Au sein d'un même groupe, Kafka assigne chaque partition à **exactement un** membre – ce qui permet la parallélisation horizontale. Plusieurs groupes différents sur le même topic reçoivent chacun **tout** le flux indépendamment.

Spécificité de Spark Structured Streaming. Spark utilise un group ID auto-généré de la forme `spark-kafka-source-<uuid>-driver-0` **par requête streaming** et *ne commit pas* les offsets dans Kafka – il les stocke dans le `checkpointLocation`, car Spark veut une sémantique exactly-once que les commits Kafka seuls ne peuvent pas garantir. Conséquence pratique : dans l'UI Kafka (`localhost:8085`), les consumer groups Spark n'apparaissent que tant que la query est active et disparaissent immédiatement après un `q.stop()`.

Inspection programmatique via `kafka-python` :

```
from kafka import KafkaAdminClient
admin = KafkaAdminClient(bootstrap_servers="kafka:9092")
groups = admin.list_consumer_groups()
for gid, _ in groups:
    offsets = admin.list_consumer_group_offsets(gid)
    for tp, meta in offsets.items():
        print(gid, tp.partition, meta.offset)
```

Listing 11 – Lister les groupes et leurs offsets

6.2. Q20 : Simulation de nouvelles données entrantes

Le producteur `02_producer_binance.ipynb` déjà construit en Part 2 sert exactement de simulateur : chaque exécution de la boucle pousse 5 nouveaux messages dans le topic

toutes les 5 s. Avec Kafka UI ouvert (`localhost:8085`, onglet *Topics* → *financial_data* → *Messages*), on observe les nouveaux messages arriver en temps réel. Sans relancer aucun consommateur, on peut faire varier dynamiquement :

- l'intervalle (`POLL_INTERVAL_SECONDS`) – pour stresser le pipeline,
- la liste des symboles (`SYMBOLS`) – pour observer le partitionnement par clé.

6.3. Q21 : Réactivité et latence du pipeline

Pour mesurer précisément la latence, on compare **trois horodatages** qui existent naturellement sur chaque message :

Horodatage	Sens
<code>event_time</code>	Champ <code>closeTime</code> renvoyé par Binance dans la réponse JSON.
<code>kafka_ts</code>	Auto-injecté par le broker quand il accepte le message.
<code>now = current_timestamp()</code>	Heure courante côté Spark au moment du micro-batch.

On en déduit deux latences indépendantes :

- `kafka_to_spark_s = now - kafka_ts` : le temps qui sépare l'acceptation par le broker de l'observation par Spark. **C'est la latence pipeline qu'on contrôle.** Mesurée à **2–4 secondes**, bornée par notre trigger de 5 s.
- `api_freshness_s = kafka_ts - event_time` : la fraîcheur des données upstream. C'est une propriété de l'API Binance, pas du pipeline. Pour le endpoint `/ticker/24hr`, le `closeTime` est l'horodatage du dernier trade dans la fenêtre rolling 24h, qui peut occasionnellement être très ancien pour des symboles peu liquides.

Sortie observée (extrait) :

symbol	price	kafka_ts	kafka_to_spark_s	api_freshness_s
BTCUSDT	78235	21:08:07.547	3	3603
ETHUSDT	2367	21:08:07.441	3	3606
SOLUSDT	86.87	21:08:07.565	3	3607

Verdict. Le pipeline est **quasi-temps réel** (*near-real-time*, sous-5 s end-to-end), mais pas *low-latency-trading* (microsecondes). Pour gagner un ordre de grandeur, on remplacerait le polling REST par un abonnement WebSocket `wss://stream.binance.com:9443/ws/btcusdt@tr` – documentation prévue mais hors scope.

6.4. Q22 : Scénarios de panne

A. Crash du broker Kafka. Simulé en arrêtant simplement le conteneur depuis l'hôte :

```
docker stop lab3-kafka          # le broker disparaît
# attendre quelques secondes -- les producteurs/consommateurs
# tentent de reconnecter
docker start lab3-kafka         # le broker revient
```

Comportement observé :

- Le producteur Spark ou kafka-python **bufferise en mémoire** et **retente** automatiquement (`retries=N`). Avec `acks=all`, aucun message n'est perdu tant que le broker revient avant saturation du buffer.
- Le consommateur Spark continue son micro-batch courant, puis reçoit une exception à la prochaine tentative de fetch ; sa logique de reprise (via `checkpointLocation`) permet de poursuivre depuis le dernier offset commité dès le retour du broker.
- **Limite de notre lab** : un seul broker, `replication-factor=1`. Si on perd la donnée disque du broker, c'est irrécupérable. En production, on utilise 3 brokers et `replication-factor=3` – la perte d'un broker est transparente.

B. Crash du job Spark. Simulé en tuant le processus Python du consommateur :

```
docker exec lab3-jupyter pkill -f spark
```

Comportement observé. Au redémarrage du même notebook, Spark inspecte le `checkpointLocation`, trouve le dernier offset Kafka commité, recharge l'état des agrégats depuis le *state store*, et reprend exactement là où il s'était arrêté. La structure du checkpoint est lisible directement :

```
/tmp/lab3_ckpt_recovery/
  metadata                (id de la query)
  offsets/0, offsets/1, ... (offsets Kafka batch par batch)
  commits/0, commits/1, ... (m\ '{e}tadonn\ '{e}es de commit)
  sources/0/0              (info sur la source)
  state/...                 (state store des agr\ '{e}gats, si
                           applicable)
```

Garanties effectives :

- **Exactly-once** pour le calcul des agrégats (state store transactionnel),
- **At-least-once** pour la livraison vers un sink non-transactionnel (console, Parquet)
 - d'où l'usage de `dropDuplicates` en aval si une garantie stricte est requise.

6.5. Note opérationnelle : reproductibilité de Kafka entre redémarrages

Au cours de nos tests, nous avons constaté que les **topics Kafka et leurs messages disparaissaient** après un `docker-compose down/up`. Deux causes à traiter pour que le prof retrouve l'environnement opérationnel à la simple commande `docker-compose up -d` :

1. **Volume persistant pour Kafka.** Sans volume monté, le répertoire `/var/lib/kafka/data` (logs et offsets) vit dans la couche écriture du conteneur, qui est détruite avec lui. Nous avons ajouté un volume Docker nommé `kafka_data` pour conserver l'état :


```
kafka:
  ...
  volumes:
    - kafka_data:/var/lib/kafka/data

volumes:
  kafka_data:
```

2. **Service kafka-init idempotent.** La création manuelle du topic via `kafka-topics -create` (Part 1, Q4) est non-reproductible. Nous avons ajouté un service « one-shot » qui attend que Kafka soit prêt puis crée le topic avec `-if-not-exists` :

```
kafka-init:
  image: confluentinc/cp-kafka:7.5.0
  depends_on: [kafka]
  command: >
    bash -c "
      until kafka-topics --bootstrap-server kafka:9092 --list \
        >/dev/null 2>&1; do sleep 2; done;
      kafka-topics --bootstrap-server kafka:9092 \
        --create --if-not-exists --topic financial_data \
        --partitions 3 --replication-factor 1 \
        --config retention.ms=604800000;
    "
  restart: "no"
```

Avec ces deux ajouts, la commande `docker-compose up -d` suffit à retrouver un environnement complet et fonctionnel après tout redémarrage.

7. PART 6 – Visualization with Superset

Pour exposer les données de notre data lake à Superset, nous avons besoin d'un *moteur SQL* qui sache lire le Parquet directement depuis HDFS. Trois options canoniques existent : **Hive Server 2**, **Trino/Presto**, et **Spark Thrift Server**. Nous avons retenu la troisième.

7.1. Q23 : Exposition des données HDFS via Spark Thrift Server

Pourquoi Spark Thrift plutôt que Hive ou Trino ?

- **Cohérence avec le reste de la stack** – nous avons déjà Spark 3.5.0 dans le projet ; le Thrift Server est livré avec la même distribution Spark, donc aucun nouveau moteur à maintenir.
- **Même dialecte SQL** que dans nos notebooks – pas de surprise quand on porte une requête du notebook vers Superset.
- **Supporte le Parquet partitionné** nativement, sans nécessiter un Hive Metastore externe (utilise un Derby embarqué).

Service Docker ajouté :

```

spark-thrift:
  image: apache/spark:3.5.0
  container_name: lab3-spark-thrift
  depends_on: [spark-master, namenode]
  ports: ["10000:10000"]
  user: root
  command: >
    bash -c "
      /opt/spark/sbin/start-thriftserver.sh \
        --master spark://spark-master:7077 \
        --hiveconf hive.server2.thrift.port=10000 \
        --hiveconf hive.server2.thrift.bind.host=0.0.0.0 \
        --conf spark.sql.warehouse.dir=hdfs://namenode:9000/lab3/
        warehouse \
        --conf spark.hadoop.fs.defaultFS=hdfs://namenode:9000 \
        && tail -f /opt/spark/logs/*"

```

Listing 12 – Service spark-thrift dans docker-compose.yml

Le Thrift Server expose le protocole **HiveServer2** sur le port 10000, accessible :

- depuis les conteneurs sur le réseau lab3 via `spark-thrift:10000`;
- depuis l'hôte via `localhost:10000` – pratique pour DBeaver, Beeline, ou tout autre client JDBC.

Vérification rapide via Beeline (le client SQL en ligne de commande livré avec Spark) :

```

docker exec lab3-spark-thrift /opt/spark/bin/beeline \
  -u "jdbc:hive2://localhost:10000" -n spark \
  -e "SHOW DATABASES;"
# +-----+
# | namespace |
# +-----+
# | default   |
# +-----+

```

Enregistrement des couches Parquet comme tables externes. Une fois le Thrift Server prêt et après avoir produit au moins quelques fichiers Parquet via le notebook 04_hdfs_medallion.ipynb, on exécute le script `scripts/setup_thrift_tables.sql` :

```

CREATE TABLE IF NOT EXISTS bronze_ticks
USING parquet
LOCATION 'hdfs://namenode:9000/lab3/bronze/ticks';

CREATE TABLE IF NOT EXISTS silver_ticks
USING parquet
LOCATION 'hdfs://namenode:9000/lab3/silver/ticks';

CREATE TABLE IF NOT EXISTS gold_moving_avg
USING parquet
LOCATION 'hdfs://namenode:9000/lab3/gold/moving_avg';

```

Listing 13 – setup_thrift_tables.sql

```
docker exec lab3-spark-thrift /opt/spark/bin/beeline \
-u "jdbc:hive2://localhost:10000" -n spark \
-f /opt/scripts/setup_thrift_tables.sql
```

Spark déduit automatiquement le schéma et les colonnes de partition à partir des fichiers Parquet déjà présents dans HDFS. Une simple vérification confirme que tout est queryable :

```
> SELECT symbol, AVG(price), COUNT(*)
   FROM silver_ticks GROUP BY symbol;
+-----+-----+-----+
| symbol | avg_price | n |
+-----+-----+-----+
| SOLUSD | 85.428    | 32 |
| ADAUSD | 0.246     | 32 |
| BNBUSD | 631.69    | 32 |
| ETHUSD | 2314.60   | 32 |
| BTCUSD | 77428.91  | 32 |
+-----+-----+-----+
```

7.2. Q24 : Configuration de Superset et création des datasets

Superset parle au Spark Thrift via SQLAlchemy + le pilote PyHive. La chaîne de connexion utilisée est :

```
hive://hive@spark-thrift:10000/default?auth=NONE
```

`auth=NONE` indique que Spark Thrift utilise SASL en mode « NONE » (pas de Kerberos). Le username `hive` est arbitraire (Spark Thrift ne vérifie pas l'authentification dans cette configuration de lab).

Plutôt que de demander au prof de cliquer manuellement dans l'UI Superset pour créer la connexion et les datasets, nous avons écrit un script `scripts/provision_superset.py` qui s'en charge via l'API REST de Superset :

```
SQLALCHEMY_URI = "hive://hive@spark-thrift:10000/default?auth=NONE"

# 1. Créer la connexion DB
requests.post(f"{SUPERSET}/api/v1/database/", json={
    "database_name": "Spark SQL",
    "sqlalchemy_uri": SQLALCHEMY_URI,
    "expose_in_sqllab": True,
}, headers={...})

# 2. Créer un dataset par table
for table in ["bronze_ticks", "silver_ticks", "gold_moving_avg"]:
    requests.post(f"{SUPERSET}/api/v1/dataset/", json={
        "database": db_id,
        "schema": "default",
```

```
"table_name": table,
}, headers={...})
```

Listing 14 – provision_superset.py (extrait)

Le script est **idempotent** : il met à jour la connexion existante et saute les datasets déjà créés. Exécution :

```
docker exec lab3-superset python /opt/scripts/provision_superset.py
# Provisioning complete.
# - Database: 'Spark SQL'
# - Datasets: bronze_ticks, silver_ticks, gold_moving_avg
```

7.3. Q25 : Construction des dashboards

Les datasets étant en place, on peut construire les graphiques directement dans l'UI Superset (localhost:8088, login admin/admin). Trois graphiques clés sont demandés par l'énoncé ; voici la configuration recommandée pour chacun :

Graphique	Type Superset	Configuration
Price evolution over time	Time-series Line Chart	Dataset : silver_ticks. Time column : event_time. Metric : AVG(price). Group by : symbol. Time grain : 1 minute.
Volume trends	Time-series Bar Chart (ou Area)	Dataset : silver_ticks. Metric : SUM(volume). Group by : symbol. Time grain : 5 minutes.
Moving averages	Time-series Line Chart	Dataset : gold_moving_avg. Time column : window_start. Metric : AVG(avg_price). Group by : symbol.

Tous trois sont ensuite assemblés dans un **Dashboard** via le bouton « + Dashboard » de l'UI, avec un filtre de symbole partagé en haut pour permettre à l'utilisateur de comparer les actifs.

7.4. Q26 : Choix du type de graphique selon l'analyse

Question analytique	Graphique recommandé	Pourquoi
Série temporelle	Time-series Line Chart	Met en évidence la continuité et la tendance ; l'œil humain suit naturellement une ligne.
Comparaison de catégories	Bar Chart (vertical ou horizontal)	Compare des valeurs discrètes côte à côte ; la longueur des barres est facile à comparer précisément.
Tendance / volume cumulé	Area Chart ou Stacked Area	La surface remplie traduit visuellement « l'accumulation » ; pratique pour une somme par catégorie au cours du temps.
Distribution	Histogram, Box Plot	Révèle l'asymétrie, les outliers, la dispersion – impossible avec une simple moyenne.
Corrélation entre 2 métriques	Scatter Plot	Visualise la relation entre prix et volume par exemple.
KPI unique	Big Number (avec ou sans trend)	Pour des métriques sentinelles affichées en grand sur un dashboard exec.

7.5. Note opérationnelle : ordre d'exécution pour reproduire la Part 6

Pour que le prof obtienne un Superset opérationnel, l'ordre suivant est requis :

1. `docker-compose up -d` – démarre tous les services dont `spark-thrift` et `superset`.
2. Lancer le notebook `02_producer_binance.ipynb` pour alimenter le topic Kafka.
3. Lancer le notebook `04_hdfs_medallion.ipynb` et le laisser tourner ~3 min pour produire au moins quelques fichiers Parquet dans `/lab3/silver/` et `/lab3/gold/`.
4. Enregistrer les tables externes côté Spark Thrift :

```
docker exec lab3-spark-thrift /opt/spark/bin/beeline \
-u "jdbc:hive2://localhost:10000" -n spark \
-f /opt/scripts/setup_thrift_tables.sql
```

5. Provisionner Superset (datasets + connexion) :

```
docker exec lab3-superset \
python /opt/scripts/provision_superset.py
```

Après ces 5 étapes, on peut ouvrir `localhost:8088` et construire les dashboards directement.

8. PART 7 – Multidimensional Analysis (Advanced)

Le notebook `notebooks/07_multidimensional_analysis.ipynb` contient toutes les requêtes et constructions de cette partie. Toutes s'exécutent contre la couche `silver_ticks` déjà produite en Part 4.

8.1. Q27 : Prix moyen par symbole et par jour

```
SELECT ingest_date,
       symbol,
       ROUND(AVG(price), 6) AS avg_price,
       COUNT(*)           AS n_ticks
FROM silver_ticks
GROUP BY ingest_date, symbol
ORDER BY ingest_date, symbol;
```

Résultat observé :

ingest_date	symbol	avg_price	n
2026-04-23	ADAUSDT	0.245959	32
2026-04-23	BNBUSDT	631.693125	32
2026-04-23	BTCUSDT	77428.907813	32
2026-04-23	ETHUSDT	2314.602187	32
2026-04-23	SOLUSDT	85.428125	32

8.2. Q28 : Symbole le plus volatile

Comparer directement l'écart-type du prix entre BTC (~\$77000) et ADA (~\$0.25) n'a pas de sens (les deux sont sur des échelles incomparables). On utilise donc le **coefficient de variation** ($CV = \text{écart-type} / \text{moyenne}$, exprimé en %) qui normalise la dispersion par le niveau de prix :

```
SELECT symbol,
       ROUND(AVG(price), 6)           AS mean_price,
       ROUND(STDDEV_SAMP(price), 6)   AS stddev_price,
       ROUND(STDDEV_SAMP(price)
             / AVG(price) * 100, 4)   AS cv_pct
FROM silver_ticks
GROUP BY symbol
ORDER BY cv_pct DESC;
```

symbol	mean_price	stddev_price	cv_pct
SOLUSDT	85.428125	0.035235	0.0412
ADAUSDT	0.245959	8.7E-5	0.0356
BTCUSDT	77428.907813	25.722947	0.0332
ETHUSDT	2314.602187	0.745463	0.0322
BNBUSDT	631.693125	0.147549	0.0234

Sur cet échantillon (couvrant quelques minutes seulement), **SOLUSDT** a le CV le plus élevé, donc le mouvement de prix relatif le plus prononcé. Sur un échantillon plus long les classements peuvent changer ; la méthodologie reste la même.

8.3. Q29 : Heures de pointe (peak trading hours)

```
SELECT HOUR(event_time)           AS hour_utc ,
       COUNT(*)                  AS n_ticks ,
       ROUND(SUM(volume), 2)     AS total_volume
FROM silver_ticks
GROUP BY HOUR(event_time)
ORDER BY total_volume DESC;
```

Sur des données de 24h, ce graphique révélerait typiquement deux « bosses » correspondant aux ouvertures successives des marchés asiatiques, européens, puis américains. Sur l'échantillon court de notre lab, l'heure 11 UTC domine simplement parce que le test a tourné pendant cet intervalle.

8.4. Q30 : ROLLUP et CUBE

Spark SQL implémente les deux opérateurs OLAP standard pour produire des **sous-totaux multi-niveaux** en une seule passe.

ROLLUP – hiérarchie ordonnée. ROLLUP(ingest_date, symbol) génère trois niveaux : par (date, symbole), par (date) (toutes paires d'un jour confondues), et le **grand total** (toutes lignes). Les niveaux agrégés ont des NULL dans les colonnes « rollées ».

```
SELECT ingest_date , symbol , ROUND(SUM(volume), 2) AS total_volume
FROM silver_ticks
GROUP BY ROLLUP(ingest_date , symbol)
ORDER BY ingest_date NULLS LAST , symbol NULLS LAST;
```

ingest_date	symbol	total_volume	
2026-04-23	ADAUSDT	3.623403051E9	<- niveau (date, sym)
2026-04-23	BNBUSDT	3990725.87	
2026-04-23	BTCUSDT	588691.8	
2026-04-23	ETHUSDT	1.200019716E7	
2026-04-23	SOLUSDT	8.308069275E7	
2026-04-23	NULL	3.72306335858E9	<- sous-total par jour
NULL	NULL	3.72306335858E9	<- grand total

CUBE – toutes les combinaisons. CUBE(ingest_date, symbol) ajoute le niveau « par symbole, tous jours confondus » en plus de ce que produit ROLLUP. C'est utile quand l'utilisateur veut pivoter dans n'importe quelle direction sans avoir à relancer la requête.

ingest_date	symbol	total_volume	
2026-04-23	ADAUSDT	3.623403051E9	
2026-04-23	...	...	
NULL	ADAUSDT	3.623403051E9	<- ADA tous jours
NULL	BNBUSDT	3990725.87	

```
| NULL      | BTCUSDT | 588691.8      |
| NULL      | ETHUSDT | 1.200019716E7 |
| NULL      | SOLUSDT | 8.308069275E7 |
| NULL      | NULL    | 3.72306335858E9 |    <- grand total
+-----+-----+-----+
```

Cas d'usage pratique : une dashboard « finance » qui doit afficher *simultanément* le total par symbole, le total par jour, et la grande somme – sans relancer trois requêtes.

8.5. Q31 : Modélisation en étoile (star schema)

Pour des analyses BI à grande échelle, on éclate `silver_ticks` en une **table de faits étroite et longue** et plusieurs **tables de dimensions courtes et larges** :

Table	Rôle
fact_ticks	Faits / mesures : price, volume, plus 3 clés étrangères (time_key, symbol_key, market_key). Partitionnée par ingest_date. Une ligne = un tick.
dim_time	Calendrier : time_key (timestamp), year, month, day, hour, minute, weekday.
dim_symbol	Description des actifs : symbol_key, base_asset (ex. BTC), quote_asset (ex. USDT).
dim_market	Description du marché : market_key, exchange, venue_type, settlement_currency. Une seule ligne dans notre lab car nous ne consommons que Binance Spot.

Construction. La fact table est obtenue en remplaçant les colonnes descriptives de `silver_ticks` par leurs clés étrangères :

```
fact_ticks = silver.select(
  col("event_time").alias("time_key"),          # FK -> dim_time
  col("symbol").alias("symbol_key"),            # FK -> dim_symbol
  lit("BINANCE_SPOT").alias("market_key"),      # FK -> dim_market
  col("price"),
  col("volume"),
  col("ingest_date"),
)
fact_ticks.write.mode("overwrite") \
  .partitionBy("ingest_date") \
  .parquet("hdfs://namenode:9000/lab3/gold_star/fact_ticks")
```

Les trois dimensions sont construites une seule fois par `distinct` ou par création manuelle (pour `dim_market`).

Pourquoi cette décomposition ?

- **Stockage** – la fact table est large (millions de lignes à terme) ; en gardant les descriptions dans des dimensions courtes (5 lignes pour `dim_symbol`), on évite de répéter `base_asset = BTC` sur chacune des millions de lignes BTC.

- **Performance des jointures** – les clés sont des entiers ou des chaînes courtes ; les dimensions tiennent entièrement en mémoire (broadcast join) ; les filtres sur `ingest_date` éliminent des partitions entières.
- **Flexibilité** – ajouter un attribut à un symbole (par exemple un secteur, un « market cap rank ») ne touche que `dim_symbol`, pas la fact table.
- **Standard BI** – Superset, Tableau, Power BI sont tous conçus autour de ce schéma.

Requête étoile typique. Volume total par actif sous-jacent, par jour :

```
SELECT f.ingest_date ,
       s.base_asset ,
       ROUND(SUM(f.volume), 2) AS total_volume ,
       COUNT(*)                AS n_ticks
FROM fact_ticks f
JOIN dim_symbol s ON f.symbol_key = s.symbol_key
GROUP BY f.ingest_date , s.base_asset
ORDER BY f.ingest_date , total_volume DESC;
```

Résultat :

ingest_date	base_asset	total_volume	n
2026-04-23	ADA	3.623403051E9	32
2026-04-23	SOL	8.308069275E7	32
2026-04-23	ETH	1.200019716E7	32
2026-04-23	BNB	3990725.87	32
2026-04-23	BTC	588691.8	32

Toutes les tables étoile sont également enregistrées dans Spark Thrift via `setup_thrift_tables.sql` (mis à jour pour inclure `fact_ticks`, `dim_time`, `dim_symbol`, `dim_market`), donc directement utilisables depuis Superset.

8.6. Note opérationnelle : MSCK REPAIR pour les tables partitionnées

Lorsqu'une table externe Spark SQL est créée avec `USING parquet LOCATION '...'` et que les fichiers Parquet ont été écrits avec `partitionBy(...)`, Spark Thrift voit la table mais retourne **0 ligne** tant que les partitions n'ont pas été découvertes. Il faut exécuter une fois :

```
MSCK REPAIR TABLE fact_ticks;
```

Cette ligne a été ajoutée pour chaque table partitionnée dans `setup_thrift_tables.sql`, ce qui assure que le prof obtient des données queryables dès la première exécution du script.

9. Conclusion

Ce lab nous a permis de concevoir et implémenter un pipeline **streaming** complet, en réutilisant les briques du Lab 2 (HDFS, Spark, Medallion) et en y ajoutant la dimension temps réel :

- **Ingestion (Part 2)** – producteur Spark qui poll Binance REST toutes les 5 s et publie en JSON sur Kafka.
- **Stream processing (Part 3)** – consommateur Spark Structured Streaming avec schéma strict, filtre d'outliers paramétré par symbole, moyenne mobile fenêtrée et agrégation par symbole.
- **Stockage (Part 4)** – trois sinks Parquet partitionnés sur HDFS implémentant l'architecture Medallion (Bronze / Silver / Gold), chacun avec son checkpoint.
- **Résilience (Part 5)** – mesure de la latence end-to-end ($\sim 2\text{--}4$ s), stratégies de tolérance aux pannes côté producteur (acks, retries) et côté consommateur (checkpointLocation).
- **Visualisation (Part 6)** – Spark Thrift Server expose le data lake en SQL HiveServer2; Superset s'y connecte via PyHive et consomme les datasets.
- **Analyse multidimensionnelle (Part 7)** – requêtes business (volatilité, peak hours), opérateurs OLAP (ROLLUP, CUBE), et modélisation en étoile matérialisée dans `/lab3/gold_star/`.

Le tout est packagé dans un `docker-compose.yml` de 12 services qui démarre en une commande, avec un volume Kafka persistant et un service `kafka-init` idempotent qui assure que la commande `docker-compose up -d` suffit toujours à obtenir un environnement opérationnel.